Univerzitet u Zenici
Politehnički fakultet

BILJEŠKE S PREDAVANJA · SEDMICA 6

Pohrana i baze podataka kao kod

Podaci kao kod: tri vrste pohrane (block, file i object storage), PostgreSQL s ulogama, privilegijama i shemom iz datoteka u Gitu, replikacija, te backup politika s mjerenjem RPO i RTO i testom restore-a. Svaki primjer radi lokalno, a ishod se provjerava testom.

PREDMET	Računarstvo u oblaku
STUDIJ	II ciklus studija, III semestar, akademska godina 2026/27
NASTAVNIK	V. prof. dr. Đulaga Hadžić
ASISTENT	ass. Mr. Adin Jahić
TERMIN	petak, 20.11.2026, 17:15–20:15, Kabinet 3204
TEMA	Pohrana i baze podataka kao kod
KOD	github.com/Dadyze/RuO-2026-27 (mape `s6/p1` do `s6/p4` i `s6/okruzenje.sh`)

OKVIR NASTAVNE JEDINICE

Ciljevi i način rada

Do sada su sve mašine i servisi bili zamjenjivi: VM se uništi i ponovo napravi iz koda, kontejner se obnovi iz slike, konfiguracija se ponovo primijeni Ansibleom. Podaci su izuzetak. Narudžbe, korisnički računi i datoteke ne nastaju iz koda nego iz rada sistema, pa se ne mogu "ponovo primijeniti". Zato se kod podataka kodom opisuje sve *oko* njih: gdje i kako se čuvaju, ko im smije pristupiti, kako se kopiraju i kako se vraćaju. Ova sedmica uvodi upravo to.

Svi primjeri rade na računaru studenta, bez računa u oblaku. Block storage se vježba na loop uređaju (datoteka koju jezgra vidi kao disk), object storage na **Garageu** (S3-kompatibilan server od jednog binarnog fajla), a baza je **PostgreSQL**. Garage i PostgreSQL rade kao obični procesi korisnika na adresi `127.0.0.1`, s podacima u direktoriju `s6/.lab`, pa za njih ne treba `sudo`.

Nakon nastavne jedinice student je u stanju:

- objasniti razliku između block, file i object storagea i za dati slučaj izabrati odgovarajući,
- napraviti, prikačiti, formatirati i proširiti blok volumen i objasniti šta od toga radi oblak, a šta operativni sistem,
- pokrenuti S3-kompatibilan object storage, napraviti bucket i ključ s najmanjim pravima i raditi s objektima kroz S3 API (boto3),
- opisati PostgreSQL uloge, shemu i privilegije SQL kodom koji je idempotentan i provjeriti ga testom,

- postaviti repliku (streaming replikacija) i objasniti zašto replikacija nije backup,
- definisati backup politiku (RPO, RTO, retencija, pravilo 3-2-1), napraviti test restore-a i izmjeriti postignuti RPO i RTO.

Okruženje i pokretanje

Alate za ovu sedmicu (PostgreSQL, `python3-boto3` i Garage) instalira `00-setup.sh`. Sistemsku instancu PostgreSQL-a skripta gasi, jer primjeri pokreću vlastite instance. Ako je skripta već pokrenuta ranije, pokrene se ponovo (idempotentna je):

```bash
cd ~/RuO-2026-27 && git pull
./00-setup.sh                    # dodaje postgresql, python3-boto3 i garage (/usr/local/bin/garage)
make check                       # psql, garage i losetup moraju biti "ok"
make s6-p1                       # pregled diskova; zatim s6-p2 ... s6-p4, opcionalno s6-p1-lab, s6-p3-replika, s6-p4-restore
```

RAD BEZ TROŠKOVA

Jedini cilj koji traži `sudo` je `s6-p1-lab` (prikačivanje i montiranje diska), i on dira samo datoteku `/var/tmp/ruo6-disk.img` i direktorij `/mnt/ruo6`. Ciljevi primjera 2 do 4 pokreću procese korisnika na portovima `3900` – `3903` (Garage) i `5461` – `5463` (PostgreSQL), samo na `127.0.0.1`. Lozinke i tajne se generišu pri prvom pokretanju u `s6/.lab` i nisu u Gitu. Sve se briše sa `make s6-clean`.

REDOSLIJED

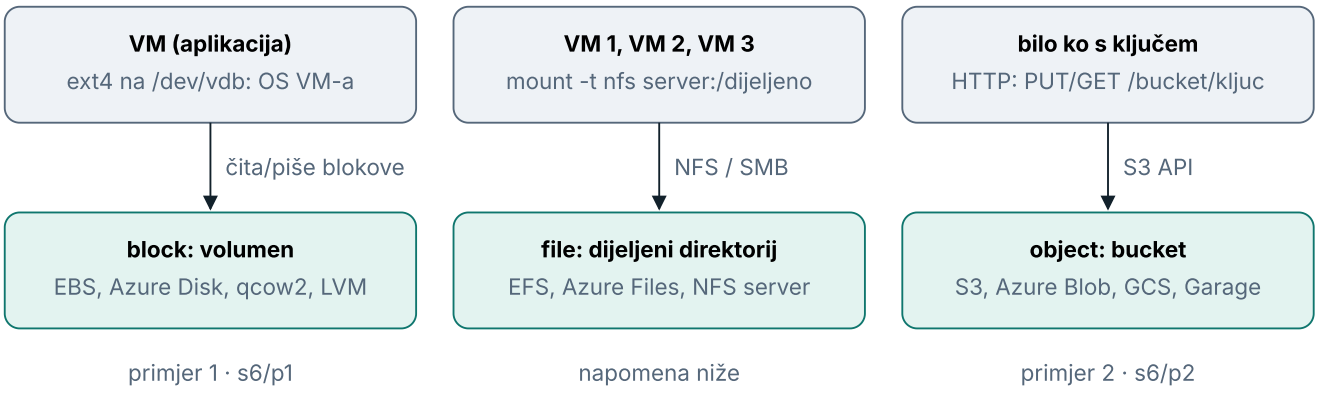
Primjer 4 pravi backup baze iz primjera 3 (ciljevi `s6-p4` i `s6-p4-restore` je sami podižu ako ne radi). Kopija backupa ide i u bucket iz primjera 2 ako Garage radi, pa je najbolje redom: `s6-p2`, `s6-p3`, `s6-p3-replika`, `s6-p4`, `s6-p4-restore`.

---

1 · POJAM I POZADINA

Tri vrste pohrane: block, file i object

Svaki oblak nudi iste tri vrste pohrane, jer svaka odgovara na drugo pitanje: *koje blokove na disku* (block), *koju datoteku na kojoj putanji* (file) ili *koji objekat pod kojim ključem* (object). Razlikuju se po tome ko upravlja datotečnim sistemom, koliko klijenata istovremeno smije pristupiti i preko čega.



Slika 1. Block storage daje sirove blokove jednoj mašini, a datotečni sistem pravi njen OS. File storage daje gotov datotečni sistem preko mreže, pa ga dijeli više mašina. Object storage nema datotečni sistem: objekti se čitaju i pišu cijeli, preko HTTP API-ja.

	BLOCK	FILE	OBJECT
Jedinica	blok (npr. 4 KiB) na adresi	datoteka na putanji	objekat pod ključem, s metapodacima
Pristup	jedna mašina (prikačen disk)	više mašina istovremeno	bilo ko s ključem, preko mreže
Izmjena	bilo koji bajt, na mjestu	bilo koji bajt, uz zaključavanje	samo cijeli objekat (novi PUT)
Kašnjenje	najmanje (ispod 1 ms)	malo (mreža + server)	veće (HTTP zahtjev po objektu)
Veličina	do nekoliko desetina TiB po volumenu	do PiB	praktično neograničena
Tipična upotreba	sistemski disk, baza podataka	dijeljeni direktoriji, stare aplikacije	backupi, slike, logovi, statički web, data lake

POZADINA: FILE STORAGE I NFS

File storage je server koji izvozi direktorij, a klijenti ga montiraju kao lokalni. Na Linuxu je to **NFS**: server u `/etc/exports` navodi šta izvozi i kome, npr. `/srv/dijeljeno 10.40.1.0/24(rw,sync,root_squash)`, a klijent ga montira naredbom `mount -t nfs 10.40.1.5:/srv/dijeljeno /mnt/dijeljeno` (ili redom u `/etc/fstab`, odnosno Ansible modulom `ansible.posix.mount`). Opcija `root_squash` root-a s klijenta pretvara u neprivilegovanog korisnika, a `sync` potvrđuje upis tek kad je na disku servera. Windows koristi **SMB**. U oblaku su to AWS EFS i Azure Files: usluga na koju se klijent spaja istim protokolom (NFS ili SMB). U ovoj sedmici NFS se ne pokreće, jer NFS server u jezgri ne radi u WSL2; primjer je ostavljen za zadatak na Linux VM-u iz sedmice 2.

ZAŠTO NE DIJELITI BLOK VOLUMEN

Datotečni sistem kao ext4 pretpostavlja da je on jedini koji piše po disku: drži dio metapodataka u memoriji i ne zna za tuđe izmjene. Ako isti blok volumen montiraju dvije mašine, svaka pokvari ono što je druga napisala. Zato oblak blok volumen zadano prikačuje samo jednoj VM, a za dijeljenje postoje file storage ili posebni klaster datotečni sistemi (GFS2, OCFS2).

2 · PRIMJER 1 · S6/P1

Block storage: volumen, datotečni sistem i proširenje

> **POZADINA PRIMJERA**
> U oblaku volumen nastaje API pozivom (Terraform resurs `aws_ebs_volume` ili `libvirt_volume` iz sedmice 2), a zatim se prikači VM-u, u kojoj se pojavi kao novi disk (`/dev/vdb`, `/dev/nvme1n1`). Sve dalje radi operativni sistem VM-a: pravi datotečni sistem, montira ga i, kad se volumen poveća, proširuje datotečni sistem. Primjer to radi lokalno: datoteka `/var/tmp/ruo6-disk.img` je volumen, `losetup` je "prikačivanje", a `/dev/loopN` je disk koji VM vidi.

bash — Isječak 1 · s6/p1/disk.sh: vježba (izvod)

```
truncate -s 256M /var/tmp/ruo6-disk.img       # novi volumen (rijetka datoteka: thin
provisioning)
losetup -f --show /var/tmp/ruo6-disk.img      # attach: datoteka postaje blok uređaj /dev/loopN
mkfs.ext4 -q -L ruo6-podaci /dev/loopN        # datotečni sistem pravi OS, ne oblak
mount /dev/loopN /mnt/ruo6

umount /mnt/ruo6 && losetup -d /dev/loopN    # detach: podaci ostaju na volumenu
losetup -f --show ... && mount ...           # attach (u oblaku: drugoj VM) i podaci su tu

truncate -s 512M /var/tmp/ruo6-disk.img       # oblak: povećanje volumena
losetup -c /dev/loopN                         # OS: uređaj je sada veći
resize2fs /dev/loopN                          # OS: datotečni sistem zauzima novi prostor
```

bash — Isječak 2 · pokretanje

```
make s6-p1           # lsblk, df -hT, findmnt: samo čita
make s6-p1-lab       # [sudo] cijela vježba; PAUZA=1 make s6-p1-lab čeka Enter nakon svakog
koraka
make s6-p1-clean     # otkači i obriši volumen
```

izlaz make s6-p1-lab (skraćeno)

```
== 1. Novi volumen od 256 MB: datoteka je 'rijetka' (sparse), pa na disku još ne zauzima
prostor
$ ls -lsh /var/tmp/ruo6-disk.img
0 -rw-r--r-- 1 root root 256M Nov 13 15:38 /var/tmp/ruo6-disk.img

== 2. Prikačivanje (attach): datoteka postaje blok uređaj /dev/loopN, bez datotečnog sistema
$ losetup -f --show /var/tmp/ruo6-disk.img
/dev/loop0
NAME  TYPE  SIZE FSTYPE MOUNTPOINTS
loop0 loop  256M

== 3. Datotečni sistem (mkfs.ext4) i montiranje: tek sada postoje direktoriji i datoteke
$ df -hT /mnt/ruo6
Filesystem     Type  Size  Used Avail Use% Mounted on
/dev/loop0     ext4  224M   28K  206M   1% /mnt/ruo6
$ ls -lsh /var/tmp/ruo6-disk.img
17M -rw-r--r-- 1 root root 256M Nov 13 15:38 /var/tmp/ruo6-disk.img

== 4. Otkačivanje i ponovno prikačivanje (detach/attach): podaci žive na volumenu, ne na 'VM-u'
$ cat /mnt/ruo6/narudzbe.txt
narudzba 1001

== 5. Proširenje volumena bez gašenja: veći uređaj, pa veći datotečni sistem (resize2fs,
online)
$ lsblk -o NAME,SIZE /dev/loop0
loop0  512M
   (uređaj je 512M, ali datotečni sistem još zna samo za 256M)
$ resize2fs /dev/loop0
The filesystem on /dev/loop0 is now 131072 (4k) blocks long.
$ df -h /mnt/ruo6
/dev/loop0      464M   28K  428M   1% /mnt/ruo6
```

Tri detalja iz izlaza vrijedi zapamtiti. Prvo, nova datoteka od 256 MB na disku zauzima 0 bajta, a nakon `mkfs` 17 MB: to je **thin provisioning**, a isto radi i qcow2 iz sedmice 2. Pružatelj zato može prodati više prostora nego što fizički ima, pa se kod njega prati stvarno zauzeće. Drugo, od 256 MB datotečnom sistemu ostaje 224 MB, a korisniku 206 MB: ext4 dio prostora troši na tabelu inode-a i dnevnik (journal), a 5% čuva za root-a. Treće, proširenje ima dva koraka i dva vlasnika: veći volumen daje oblak, a veći datotečni sistem pravi OS. Ako se drugi korak zaboravi, disk je veći, a `df` i dalje javlja da je pun.

> **ONLINE I OFFLINE PROŠIRENJE**
> ext4 i XFS se mogu proširiti dok su montirani (`resize2fs` , odnosno `xfs_growfs`). Neka okruženja (kontejneri, stariji WSL kerneli) to ne dozvoljavaju, pa skripta tada kratko otkači datotečni sistem, provjeri ga (`e2fsck -f`) i proširi offline. Smanjivanje je drugačije: XFS se uopšte ne može smanjiti, a ext4 samo offline, pa se volumen u praksi samo povećava. Snimak volumena (*snapshot*) kod pružatelja je kopija svih blokova u jednom trenutku i dobar je kao brza tačka povratka, ali za bazu koja radi daje kopiju "kao nakon nestanka struje" (vidi primjer 4).

Object storage: S3 API s Garageom

POZADINA PRIMJERA

S3 (Simple Storage Service) je AWS-ova usluga iz 2006, a njen HTTP API je postao standard: isti klijent radi s AWS S3, Azure Blobom preko prolaza, Ceph RGW, Garageom ili MinIO-om, i mijenja se samo adresa servera (*endpoint*). Prostor je ravan: **bucket** sadrži **objekte**, a objekat je ključ (npr. `izvjestaji/2026/11/novembar.txt`), sadržaj i metapodaci. Direktorija nema: kosa crta je dio ključa, a "direktorij" je samo zajednički prefiks. Svaki zahtjev je potpisan tajnim ključem (AWS Signature V4), pa server zna ko pita i provjerava njegova prava na bucket.

ZAŠTO GARAGE, A NE MINIO

Silabus spominje MinIO, koji je godinama bio zadani izbor za lokalni S3. MinIO, Inc. je u oktobru 2025. prestao objavljivati binarne verzije i Docker slike izdanja zajednice (zadnje slike imaju poznatu ranjivost), a u 2026. su repozitoriji `minio/minio` i `minio/mc` nestali s Docker Huba. Garage (Deuxfleurs, AGPLv3) je S3-kompatibilan server pisan u Rustu, jedan statički binarni fajl, napravljen za klastere na više lokacija. Za klijenta je razlika nevidljiva: kod iz ovog primjera bez izmjena radi i s MinIO-om i s AWS S3.

toml — Isječak 3 · s6/p2/garage.toml (izvod)

```toml
metadata_dir = "@DIR@/meta"          # metapodaci: buckets, ključevi, popis objekata (mala baza)
data_dir     = "@DIR@/data"          # sami objekti, podijeljeni u blokove
replication_factor = 1               # lab: jedna kopija. Produkcija: 3 (svaki blok u 3 zone)
rpc_secret   = "@RPC_SECRET@"        # zajednička tajna čvorova (32 bajta, hex)

[s3_api]
s3_region     = "garage"
api_bind_addr = "127.0.0.1:3900"     # S3 API: ovdje se spajaju klijenti (boto3, aws cli, rclone)
```

Skripta `objekti.sh` zamijeni `@...@` vrijednosti (tajne generiše `openssl rand`), pokrene server i zatim kodom opiše ostatak. Garage je klaster i prije prvog upisa mora znati **layout**: koji čvor drži koliko podataka i u kojoj zoni. Tek onda se prave bucket i ključ, a ključ dobija prava samo na taj jedan bucket.

bash — Isječak 4 · s6/p2/objekti.sh (izvod)

```bash
garage layout assign -z dc1 -c 1G "$NODE"    # čvor u zoni dc1, kapacitet 1 GB
garage layout apply --version 1              # layout ima verziju: promjena klastera je eksplicitna
garage bucket create ruo-backup
garage key create ruo-backup-kljuc
garage bucket allow --read --write --owner ruo-backup --key ruo-backup-kljuc
# ID i tajna ključa idu u s6/.lab/garage/s3.env (chmod 600), odakle ih čitaju s3.py i backup.sh
```

python — Isječak 5 · s6/p2/s3.py (izvod)

```python
s3 = boto3.client("s3", endpoint_url=os.environ["S3_ENDPOINT"], region_name="garage",
                  config=Config(s3={"addressing_style": "path"}))   # http://host:3900/bucket/kljuc

s3.put_object(Bucket=b, Key="izvjestaji/2026/11/novembar.txt", Body=tijelo,
```

```
              ContentType="text/plain", Metadata={"autor": "ruo", "sedmica": "6"})
s3.list_objects_v2(Bucket=b, Prefix="izvjestaji/")              # "direktorij" = prefiks
s3.list_objects_v2(Bucket=b, Delimiter="/")                    # prvi nivo: CommonPrefixes
s3.head_object(Bucket=b, Key=k)                                # metapodaci bez sadržaja
url = s3.generate_presigned_url("get_object", Params={"Bucket": b, "Key": k}, ExpiresIn=60)
```

bash — Isječak 6 · pokretanje

```
make s6-p2          # objekti.sh up (konfiguracija, server, layout, bucket, ključ) + s3.py
demo
make s6-p2-clean    # gasi Garage i briše podatke
```

izlaz make s6-p2: koraci 2 do 4 (server, layout, bucket i ključ)

```
== 2. Server (garage server) u pozadini
  pokrenut (pid 25587), dnevnik: s6/.lab/garage/server.log
== 3. Layout klastera: koji čvor drži koliko podataka i u kojoj zoni
$ garage layout assign -z dc1 -c 1G
736a30c30f51bbdf63b8d2770d6d74d0bc7e2a336d8f6852962469cfe057032b
$ garage layout apply --version 1
  ID                Hostname  Address         Tags  Zone  Capacity   DataAvail
ResyncQueue  Version
  736a30c30f51bbdf  vm        127.0.0.1:3901  []    dc1   953.7 MiB  29.3 GiB (11.6%)  0
v2.5.0
== 4. Bucket i pristupni ključ s pravima samo na taj bucket (načelo najmanjih prava)
$ garage bucket create ruo-backup
$ garage key create ruo-backup-kljuc
$ garage bucket allow --read --write --owner ruo-backup --key ruo-backup-kljuc
  Permissions  Access key                                  Local aliases
  RWO          GKa3d23b0583b648d69cb7f972  ruo-backup-kljuc
  ključ GKa3d23b0583b648d69cb7f972 -> s6/.lab/garage/s3.env (tajna se ne ispisuje)
```

izlaz make s6-p2: S3 klijent (s3.py demo)

```
== 2. PUT: objekat = ključ + bajtovi + metapodaci (nema direktorija ni djelimičnog pisanja)
  PUT s3://ruo-backup/izvjestaji/2026/11/novembar.txt
  PUT s3://ruo-backup/izvjestaji/2026/10/oktobar.txt
  PUT s3://ruo-backup/slike/logo.txt
== 3. LIST s prefiksom: 'direktorij' je samo zajednički početak ključa
  prefiks 'izvjestaji/':
  2026-11-13 14:56:22        39 B  izvjestaji/2026/10/oktobar.txt
  2026-11-13 14:56:22        39 B  izvjestaji/2026/11/novembar.txt
  prvi nivo (Delimiter='/'): ['izvjestaji/', 'slike/']
== 4. HEAD: metapodaci bez preuzimanja sadržaja; ETag je MD5 sadržaja (za obični PUT)
  ContentLength=39  ContentType=text/plain  Metadata={'autor': 'ruo', 'sedmica': '6'}
  ETag="99a794363f32bb33a7487f215b447db6"   md5 lokalno="99a794363f32bb33a7487f215b447db6"
== 5. Izmjena = novi PUT cijelog objekta (stari sadržaj se zamjenjuje, ne dopisuje)
  GET: narudzbe za novembar: 1001, 1002, 1003 | 1004 |
== 6. Presigned URL: privremeni link bez ključa (vrijedi 60 s), npr. za preuzimanje iz browsera
   http://127.0.0.1:3900/ruo-backup/slike/logo.txt?X-Amz-...&X-Amz-Expires=60&X-Amz-
Signature=...
   GET s potpisanim URL-om: HTTP 200
   isti URL bez potpisa:      HTTP 403 (bucket nije javan)
== 7. Pogrešan ključ: zahtjev je potpisan (AWS SigV4), pa server odbija nepoznat ključ
   AccessDenied: 403
```

Pokretanje je idempotentno: drugi `make s6-p2` javlja "već radi" i "layout već postoji", ne pravi novi bucket ni ključ, a `bucket allow` samo potvrdi postojeća prava. Vrijeme u popisu objekata je u UTC-u, kao i

svugdje u S3 API-ju.

> **PRESIGNED URL**
> Potpisani URL nosi potpis i rok važenja u samom linku (`X-Amz-Signature`, `X-Amz-Expires`). Ko ima link, smije uraditi tačno tu operaciju na tom objektu do isteka roka, bez ključa. Tako aplikacija korisniku daje preuzimanje velike datoteke direktno iz storagea, umjesto da je prosljeđuje kroz svoj server, a bucket ostaje privatan. Javni bucketi su čest uzrok curenja podataka, pa ih oblaci zadano zabranjuju (AWS: *Block Public Access*).

> **ISTI POSAO U OBLAKU, KAO TERRAFORM**
> Na AWS-u bi bucket, zabrana javnog pristupa, verzionisanje i pravilo životnog ciklusa bili resursi: `aws_s3_bucket`, `aws_s3_bucket_public_access_block`, `aws_s3_bucket_versioning` (stara verzija objekta se čuva nakon prepisivanja ili brisanja, pa greška nije konačna) i `aws_s3_bucket_lifecycle_configuration` (npr. backupe nakon 30 dana prebaci u jeftiniju klasu, a nakon 365 obriši). Ključ s pravima samo na taj bucket je IAM politika. Pristup je isti kao u ovom primjeru: zadano sve zatvoreno, a prava se daju po bucketu i po operaciji.

Baza podataka kao kod

Baza ima dva dijela koja se ponašaju potpuno različito. **Struktura** (instanca i njene postavke, uloge, baze, sheme, tabele, privilegije, pravila pristupa) se piše unaprijed, mijenja rijetko i namjerno, i pripada Gitu, kao i sve do sada. **Podaci** nastaju u radu i ne mogu se izvesti iz koda; za njih postoje replikacija i backup. Kad je struktura kod, nova instanca (za test, za repliku, za restore) dobija identične uloge i prava jednom naredbom, umjesto da ih neko ručno "klikne" i zaboravi pola.

DIO	GDJE JE U PRIMJERU	U PRAKSI JOŠ
postavke instance	`s6/p3/postgresql.ruo.conf`	Ansible rola, parametar grupa kod RDS-a (`aws_db_parameter_group`)
ko se smije spojiti i kako	`s6/p3/pg_hba.conf`	security grupa + `pg_hba`; kod RDS-a samo mreža i IAM
uloge i lozinke	`sql/01-uloge.sql` (lozinke iz `.lab`)	Terraform provider `cyrilgdn/postgresql`, Ansible `community.postgresql`, Vault
baza, shema, tabele	`sql/02-baza.sql`, `sql/03-shema.sql`	alati za migracije: Flyway, Liquibase, Alembic, Django/Rails migracije
privilegije	`sql/04-privilegije.sql`	isto kao uloge; provjera testom

> **POZADINA: WAL**
> PostgreSQL svaku izmjenu prvo upiše u **WAL** (write-ahead log), sekvencijalni dnevnik izmjena, i tek kad je WAL na disku potvrdi transakciju. Same tabele se ažuriraju kasnije. Ako server padne, pri pokretanju ponovo primijeni WAL od zadnje kontrolne tačke i ništa potvrđeno se ne gubi. Na istom dnevniku počiva sve u ovoj sedmici što nije `pg_dump`: replika prima WAL i primjenjuje ga (primjer 3), a arhiva WAL-a omogućava povratak u bilo koji trenutak (PITR, primjer 4). Pozicija u WAL-u je **LSN** (log sequence number), pa se zaostatak replike mjeri kao razlika dva LSN-a u bajtima. MySQL ima isti mehanizam pod imenom *binlog* (za replikaciju) i *redo log* (za oporavak).

U oblaku se baza najčešće uzima kao upravljana usluga: AWS RDS i Aurora, Azure Database for PostgreSQL, Google Cloud SQL. Pružatelj tada radi instalaciju, nadogradnje, replike i backupe, a korisnik i dalje kodom opisuje instancu (Terraform `aws_db_instance`: veličina, verzija, `backup_retention_period`, `multi_az`) i sve unutar baze: uloge, sheme i privilegije. Primjer 3 zato pokazuje upravo taj drugi dio, koji je isti i na vlastitom serveru i u RDS-u.

PostgreSQL: uloge, shema i privilegije iz koda, pa replika

> **POZADINA PRIMJERA**
> Baza `prodavnica` ima kupce, narudžbe i pogled sa zbirnim prometom. Pet uloga dobija tačno ono što im treba: `vlasnik` posjeduje objekte, ali se ne može prijaviti (migracije se rade sa `SET ROLE vlasnik`); `app` čita i piše, ali ne briše redove i ne mijenja strukturu; `izvjestaj` vidi samo zbirni pogled, ne i lične podatke kupaca; `repl` smije samo replikaciju; `backup` čita sve, a ne piše ništa. Administrator je OS korisnik, preko unix socketa i bez lozinke (`peer`). Skripta `baza.sh` napravi instancu (`initdb`), postavi `pg_hba.conf` i postavke iz repozitorija, pokrene je i primijeni četiri SQL datoteke redom.

pg_hba — Isječak 7 · s6/p3/pg_hba.conf

```
# TIP    BAZA          KORISNIK        ADRESA          METODA
local    all           all                             peer            # admin: OS korisnik
preko unix socketa
host     prodavnica    app,izvjestaj   127.0.0.1/32    scram-sha-256   # aplikacija i
izvještaji: samo svoja baza
host     all           backup          127.0.0.1/32    scram-sha-256   # pg_dump: čita sve
baze
host     replication   repl            127.0.0.1/32    scram-sha-256   # replika i
pg_basebackup
host     all           all             0.0.0.0/0       reject          # sve ostalo: odbijeno
```

Pravila se čitaju odozgo i vrijedi *prvo* koje odgovara tipu veze, bazi, korisniku i adresi. Zato je zadnji red "sve ostalo odbij": bez njega bi o pristupu odlučivao samo redoslijed i slučajno širok red iznad. Baza `replication` nije prava baza, nego oznaka za replikacijske veze.

sql — Isječak 8 · s6/p3/sql/01-uloge.sql (izvod)

```
SELECT 'CREATE ROLE vlasnik NOLOGIN'        WHERE NOT EXISTS (SELECT FROM pg_roles WHERE
rolname = 'vlasnik')   \gexec
SELECT 'CREATE ROLE app LOGIN'              WHERE NOT EXISTS (SELECT FROM pg_roles WHERE
rolname = 'app')       \gexec
SELECT 'CREATE ROLE repl LOGIN REPLICATION' WHERE NOT EXISTS (SELECT FROM pg_roles WHERE
rolname = 'repl')      \gexec

ALTER ROLE app       PASSWORD :'app_lozinka'       CONNECTION LIMIT 20;   -- aplikacija: čita i
piše
ALTER ROLE izvjestaj PASSWORD :'izvjestaj_lozinka' CONNECTION LIMIT 5;    -- izvještaji: samo
zbirni podaci
GRANT pg_read_all_data TO backup;                                         -- ugrađena uloga:
čita sve, ne piše
```

PostgreSQL nema `CREATE ROLE IF NOT EXISTS`, pa se idempotentnost postiže trikom: `SELECT` vrati tekst naredbe samo ako uloga ne postoji, a psql naredba `\gexec` izvrši ono što je upit vratio. Lozinke nisu u datoteci: `baza.sh` ih generiše jednom u `s6/.lab/pg/lozinke.env` i predaje kao psql varijable (`-v app_lozinka=...`), a `:'app_lozinka'` ih ubacuje kao ispravno navedeni string.

sql — Isječak 9 · s6/p3/sql/03-shema.sql i 04-privilegije.sql (izvod)

```
SET ROLE vlasnik;                                  -- objekte pravi i posjeduje vlasnik, ne
aplikacija
```

```
CREATE TABLE IF NOT EXISTS prodaja.narudzbe (
  id       bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  kupac_id bigint NOT NULL REFERENCES prodaja.kupci,
  iznos    numeric(10,2) NOT NULL CHECK (iznos > 0),
  vrijeme  timestamptz NOT NULL DEFAULT now());
CREATE OR REPLACE VIEW prodaja.v_promet_po_danu AS
  SELECT vrijeme::date AS dan, count(*) AS narudzbi, sum(iznos) AS promet FROM prodaja.narudzbe
GROUP BY 1;
RESET ROLE;

REVOKE ALL ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA prodaja TO app, izvjestaj;
GRANT SELECT, INSERT, UPDATE ON prodaja.kupci, prodaja.narudzbe TO app;    -- bez DELETE
GRANT SELECT ON prodaja.v_promet_po_danu TO izvjestaj;                     -- samo zbirni pogled
ALTER DEFAULT PRIVILEGES FOR ROLE vlasnik IN SCHEMA prodaja GRANT SELECT, INSERT, UPDATE ON
TABLES TO app;
```

bash — Isječak 10 · pokretanje

```
make s6-p3           # instanca, SQL kod dva puta (otisak stanja), test privilegija
make s6-p3-replika   # replika na 5462 i provjera replikacije
make s6-p3-clean     # gasi instance i briše podatke
```

izlaz make s6-p3: idempotentnost

```
== 1. Instanca: initdb (prazan klaster) + pg_hba.conf i postavke iz repozitorija
  initdb -> ~/RuO-2026-27/s6/.lab/pg/primarni (PostgreSQL 16)
  radi na 127.0.0.1:5461
== 2. Prva primjena SQL koda (uloge, baza, shema, privilegije)
  psql -f sql/01-uloge.sql
  psql -f sql/02-baza.sql
  psql -f sql/03-shema.sql
  psql -f sql/04-privilegije.sql
  otisak stanja: e85ef330086a
== 3. Ponovna primjena istog koda: otisak mora ostati isti (idempotentnost)
  ...
  otisak stanja: e85ef330086a
== 4. Uloge i njihova svojstva (\du)
 Role name |                         Attributes
-----------+------------------------------------------------------------
 app       | 20 connections
 backup    |
 izvjestaj | 5 connections
 repl      | Replication
 student   | Superuser, Create role, Create DB, Replication, Bypass RLS
 vlasnik   | Cannot login
```

Kod SQL-a nema ispisa `changed=0` kao kod Ansiblea, pa skripta pravi **otisak stanja**: MD5 izlaza naredbi `pg_dumpall --roles-only --no-role-passwords` i `pg_dump --schema-only`, tj. svih uloga, objekata i privilegija. Isti otisak nakon druge primjene znači da kod nije promijenio ništa. Isti otisak se može koristiti i za drift: ako neko ručno doda privilegiju, otisak se promijeni.

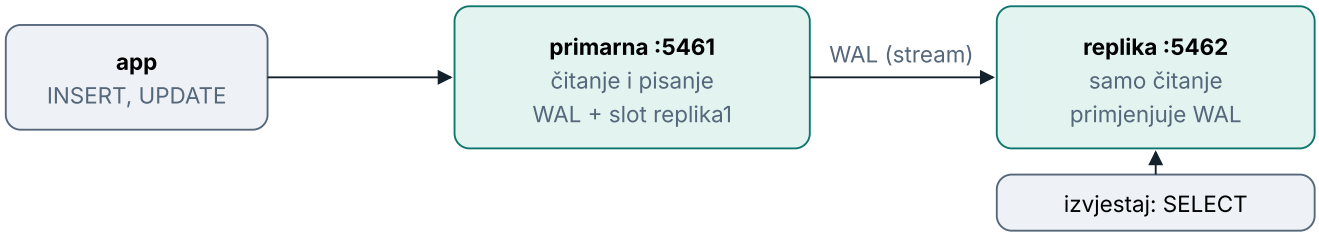
izlaz make s6-p3: test privilegija

```
== 1. app (aplikacija): čita i piše svoje tabele
  app: INSERT INTO narudzbe                  DOZVOLJENO id=1
  app: SELECT count(*) FROM narudzbe         DOZVOLJENO 1
== 2. app ne smije brisati redove ni mijenjati strukturu (nije vlasnik)
  app: DELETE FROM narudzbe                  ODBIJENO   permission denied for table narudzbe
  app: DROP TABLE narudzbe                   ODBIJENO   must be owner of table narudzbe
  app: CREATE TABLE u public shemi           ODBIJENO   permission denied for schema public
== 3. izvjestaj: vidi samo zbirni pogled, ne i kupce (lični podaci)
  izvjestaj: SELECT v_promet_po_danu         DOZVOLJENO 2026-11-13 1 25.50
  izvjestaj: SELECT email FROM kupci         ODBIJENO   permission denied for table kupci
  izvjestaj: INSERT INTO narudzbe            ODBIJENO   permission denied for table narudzbe
== 4. pg_hba.conf i lozinke: pogrešna baza, pogrešna lozinka, uloga samo za replikaciju
  app -> baza postgres (nije u pg_hba)       ODBIJENO   pg_hba.conf rejects connection for host
"127.0.0.1", user "app", database "postgres"
  app s pogrešnom lozinkom                   ODBIJENO   password authentication failed for user
"app"
  repl -> baza prodavnica                    ODBIJENO   pg_hba.conf rejects connection for host
"127.0.0.1", user "repl", database "prodavnica"
```

ZAŠTO APLIKACIJA NIJE VLASNIK TABELA

Najčešća postavka u praksi je jedna uloga za sve: aplikacija se spaja kao vlasnik baze, ili čak kao `postgres`. Tada SQL injection ili greška u kodu može obrisati tabele, a izvještajni alat vidi lične podatke. Odvojene uloge koštaju nekoliko redova koda, a šteta od kompromitovane aplikacije ostaje ograničena na ono što ta uloga smije. `ALTER DEFAULT PRIVILEGES` rješava drugi čest problem: tabela koju doda nova migracija automatski dobija ista prava, pa aplikacija ne pukne nakon deploya.

Replikacija



Slika 2. Streaming replikacija: primarna šalje WAL replici čim nastane, a replika ga primjenjuje i u isto vrijeme odgovara na upite za čitanje (hot standby). Slot `replika1` čuva WAL na primarnoj dok ga replika ne preuzme.

Replika nastaje kopijom cijele instance: `pg_basebackup` se spoji kao `repl`, preuzme sve datoteke i WAL nastao za vrijeme kopiranja, a opcija `-R` zapiše `standby.signal` i adresu primarne (`primary_conninfo`), pa se kopija pri pokretanju ponaša kao replika. Opcije `-C -S replika1` na primarnoj prave **replikacijski slot**: primarna ne briše WAL koji replika još nije preuzela, pa replika može nakratko biti nedostupna bez gubitka veze. Cijena je to što ugašena replika sa slotom puni disk primarne, pa se zaostatak prati.

izlaz make s6-p3-replika

```
== 1. Replika: pg_basebackup kopira primarnu instancu i pravi replikacijski slot
$ pg_basebackup -h 127.0.0.1 -p 5461 -U repl -D .../s6/.lab/pg/replika -R -X stream -C -S
replika1
  -R je napisao standby.signal i primary_conninfo (postgresql.auto.conf):
    primary_conninfo = 'user=repl password=***** channel_binding=prefer host=127.0.0.1
  replika radi na 127.0.0.1:5462
== 2. Primarna instanca vidi repliku (pg_stat_replication)
 application_name | client_addr |   state   | sync_state |   replay_lag
------------------+-------------+-----------+------------+-----------------
 walreceiver      | 127.0.0.1   | streaming | async      | 00:00:00.000059
  replika: pg_is_in_recovery() = t
== 3. Upis na primarnoj, čitanje na replici
  primarna: INSERT jedne narudžbe (app)
  replika:  narudžbi prije 1, sada 2
  zaostatak replike: 0 bajta WAL-a
== 4. Replika je samo za čitanje
  app: INSERT na replici                    ODBIJENO   cannot execute INSERT in a read-only
transaction
  izvjestaj: SELECT na replici              DOZVOLJENO 2
```

POZADINA: ASINHRONA I SINHRONA REPLIKACIJA, FAILOVER

Replika je `async`: primarna potvrdi transakciju čim je WAL na njenom disku, a replici ga šalje odmah poslije. Ako primarna padne u tom trenutku, posljednje transakcije su izgubljene (RPO veći od nule, obično ispod sekunde). Sa `synchronous_standby_names` primarna čeka potvrdu replike (`sync`): nema gubitka, ali svaki upis traje duže, a ako replika padne, upisi stanu. Kad primarna padne, replika se pretvara u primarnu naredbom `pg_ctl promote` (ili SQL `SELECT pg_promote()`): to je **failover**. Da to ne radi čovjek u tri ujutro, koristi se Patroni (s etcd-om bira novu primarnu i sprječava da dvije instance istovremeno misle da su primarne, tzv. *split brain*), a kod RDS-a `multi_az = true`. Aplikacija se spaja na adresu koja uvijek pokazuje na trenutnu primarnu, što je tema sedmice 7 (HAProxy, keepalived).

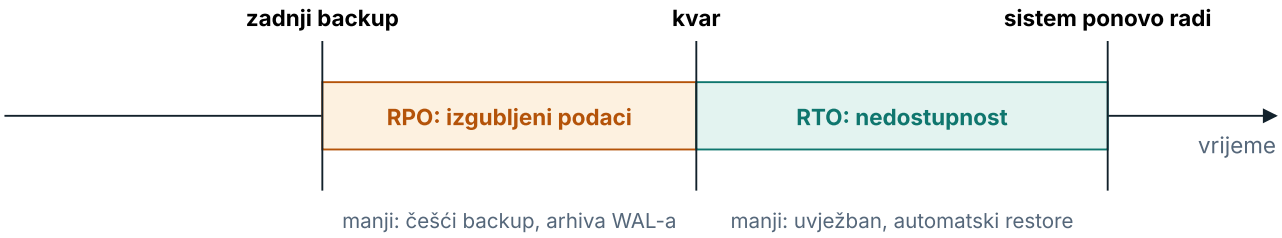
MYSQL UKRATKO

Ista ideja, drugi nazivi: uloge su korisnici vezani za adresu (`CREATE USER 'app'@'10.40.%' IDENTIFIED BY ...`, `GRANT SELECT, INSERT ON prodavnica.* TO ...`), a replika čita binlog primarne (`CHANGE REPLICATION SOURCE TO ... SOURCE_AUTO_POSITION=1` uz GTID). Logički backup je `mysqldump --single-transaction`, a fizički Percona XtraBackup ili `mysqlbackup`.

6 · POJAM I POZADINA

Backup politika: RPO, RTO i test restore-a

Backup politika odgovara na dva pitanja, a odgovori su poslovna odluka, ne tehnička: **koliko podataka smijemo izgubiti** i **koliko dugo sistem smije biti nedostupan**. Prvo je **RPO** (Recovery Point Objective), drugo **RTO** (Recovery Time Objective). Tek iz njih slijedi tehnika: koliko često backup, gdje se čuva, koliko dugo i kako se vraća.



Slika 3. RPO je razmak između zadnje kopije i kvara: sve upisano u tom razmaku je izgubljeno. RTO je razmak od kvara do ponovnog rada: otkrivanje kvara, nabavka backupa, restore, provjera i preusmjeravanje aplikacije.

POJAM	ZNAČENJE
Logički backup	`pg_dump`: SQL ili komprimovani format s naredbama koje ponovo prave objekte i podatke. Prenosiv između verzija i arhitektura, može vratiti jednu tabelu; spor za velike baze i ima RPO jednak razmaku između dumpova.
Fizički backup	`pg_basebackup` (ili alati pgBackRest, Barman, WAL-G): kopija datoteka instance. Brz za velike baze, ali vraća samo cijelu instancu, na istu glavnu verziju.
PITR	Point-in-time recovery: fizički backup + arhiva svih WAL datoteka nakon njega. Vraća se u bilo koju sekundu, npr. tačno prije pogrešnog `DROP TABLE`; RPO pada na sekunde.
Retencija	Koliko se kopija čuva. Česta šema GFS (grandfather-father-son): dnevne 7 dana, sedmične 4 sedmice, mjesečne 12 mjeseci.
Pravilo 3-2-1	Tri kopije podataka, na dvije vrste medija, od kojih je jedna izvan lokacije. Danas se dodaje i jedna kopija koja se ne može izmijeniti (*immutable*, npr. S3 Object Lock), zbog ransomwarea koji briše i backupe.
Test restore-a	Redovno vraćanje backupa u izolovanu okolinu i provjera podataka. Jedini dokaz da backup postoji i jedini način da se izmjeri stvarni RTO.

REPLIKA I SNIMAK NISU BACKUP

Replika štiti od kvara mašine, ali ne i od greške: `DROP TABLE`, pogrešan `UPDATE` bez `WHERE` ili ransomware se na repliku prenesu za milisekunde. Snimak volumena (primjer 1) je dobar dodatak, ali za bazu koja radi daje stanje "kao nakon nestanka struje" i čuva se kod istog pružatelja, često u istom računu, pa ga briše isti napadač. Backup je kopija koja je *odvojena* od izvora (vremenski, prostorno i po pravima pristupa) i za koju je provjereno da se može vratiti.

7 · PRIMJER 4 · S6/P4

Backup, retencija, test restore-a i mjerenje RPO/RTO

> **POZADINA PRIMJERA**
> Politika je datoteka u Gitu (`politika.env`), a `backup.sh` je izvršava: `pg_dump` kao uloga `backup` (samo čitanje), kontrolni zbir SHA-256 uz svaki backup, brisanje starijih od zadnjih `ZADRZI` , te kopija u bucket iz primjera 2 kao kopija izvan servera. Drugi dio skripte simulira kvar i oporavak i mjeri postignuto u odnosu na ciljeve iz politike.

bash — Isječak 11 · s6/p4/politika.env

```
BAZA=prodavnica
ZADRZI=3              # lokalno se čuva zadnjih N backupa (retencija); starije briše backup.sh
RPO_CILJ_MIN=60       # Recovery Point Objective: najviše 60 min izgubljenih podataka -> backup barem svaki sat
RTO_CILJ_MIN=15       # Recovery Time Objective: baza mora ponovo raditi najviše 15 min nakon kvara
S3_BUCKET=ruo-backup  # kopija izvan servera (Garage iz s6/p2), ako radi: pravilo 3-2-1
S3_PREFIKS=postgres/
```

bash — Isječak 12 · s6/p4/backup.sh: backup (izvod)

```
ime="$BAZA-$(date +%Y%m%d-%H%M%S).dump"
# -Fc: komprimovani "custom" format; pg_restore iz njega može vratiti sve ili samo dio (jednu tabelu)
PGPASSWORD="$(lozinka backup)" pg_dump -h 127.0.0.1 -p 5461 -U backup -Fc -f "$BK/$ime" "$BAZA"
(cd "$BK" && sha256sum "$ime" > "$ime.sha256")
# retencija: zadržati zadnjih ZADRZI, starije obrisati (ls -t: najnoviji prvi)
ls -1t "$BK"/*.dump | tail -n +$((ZADRZI + 1)) | while read -r star; do rm -f "$star" "$star.sha256"; done
s3_radi && s3 put "$S3_BUCKET" "$S3_PREFIKS$ime" "$BK/$ime"      # kopija izvan servera
```

bash — Isječak 13 · pokretanje

```
make s6-p4           # politika + 4 backupa s prometom između njih: retencija i kopija u S3
make s6-p4-restore   # kvar, test restore-a u izolovanu instancu, oporavak, RPO i RTO
make s6-p4-clean     # briše backupe
```

izlaz make s6-p4 (skraćeno: backup 1 i 4, popisi)

```
== Backup 1/4 (prije njega: promet od 5 narudžbi)
$ pg_dump -h 127.0.0.1 -p 5461 -U backup -Fc -f prodavnica-20261113-155807.dump prodavnica
  prodavnica-20261113-155807.dump  8.0K  narudžbi: 7  sha256: 33d02d718be9a09b...
  kopija izvan servera: s3://ruo-backup/postgres/prodavnica-20261113-155807.dump
...
== Backup 4/4 (prije njega: promet od 5 narudžbi)
$ pg_dump -h 127.0.0.1 -p 5461 -U backup -Fc -f prodavnica-20261113-155812.dump prodavnica
  prodavnica-20261113-155812.dump  8.0K  narudžbi: 22  sha256: 5bbc10be7bc1b255...
  retencija (ZADRZI=3): obrisan prodavnica-20261113-155807.dump
  kopija izvan servera: s3://ruo-backup/postgres/prodavnica-20261113-155812.dump
== Lokalni backupi (~/RuO-2026-27/s6/.lab/backup)
  prodavnica-20261113-155812.dump  8.0K
  prodavnica-20261113-155810.dump  8.0K
  prodavnica-20261113-155808.dump  8.0K
== Kopije u S3 (s3://ruo-backup/postgres/)
  2026-11-13 14:58:07      6683 B  postgres/prodavnica-20261113-155807.dump
  2026-11-13 14:58:07        98 B  postgres/prodavnica-20261113-155807.dump.sha256
  2026-11-13 14:58:09      6718 B  postgres/prodavnica-20261113-155808.dump
  ...
  2026-11-13 14:58:12      6791 B  postgres/prodavnica-20261113-155812.dump
  2026-11-13 14:58:12        98 B  postgres/prodavnica-20261113-155812.dump.sha256
```

Lokalno ostaju tri backupa, a u bucketu sva četiri: retencija u `backup.sh` važi samo za lokalni disk. Kopija izvan servera ima vlastitu, dužu retenciju, koju u oblaku radi pravilo životnog ciklusa bucketa (npr. brisanje objekata pod `postgres/` starijih od 90 dana), a ne skripta na serveru. To je namjerno: ključ kojim server šalje backupe u praksi smije upisivati, ali ne i brisati (ili bucket ima Object Lock), pa kompromitovan server ne može uništiti stare kopije.

izlaz make s6-p4-restore

```
== 1. Stanje prije kvara: zadnji backup i promet nakon njega
  zadnji backup: prodavnica-20261113-154716.dump
  primarna: 21 narudžbi
  + 7 novih narudžbi NAKON backupa -> primarna: 28
== 2. Kvar: ljudska greška, DROP TABLE na primarnoj instanci
$ psql -c 'DROP TABLE prodaja.narudzbe CASCADE'
  primarna: nema tabele
  replika:  nema tabele   <- replika je odmah ponovila grešku: replikacija NIJE backup
== 3. Test restore-a u izolovanu instancu (127.0.0.1:5463): da li se backup uopšte može
vratiti?
$ sha256sum -c prodavnica-20261113-154716.dump.sha256
  prodavnica-20261113-154716.dump: OK
  initdb + start: prazna instanca
  uloge nisu u pg_dump-u, pa se prvo primjenjuje kod iz s6/p3:
  psql -f sql/01-uloge.sql
$ pg_restore -C -d postgres prodavnica-20261113-154716.dump
  provjera: narudžbi 21; izvjestaj vidi promet: 21
  test restore-a trajao: 0.7 s
== 4. Oporavak primarne instance iz istog backupa
$ dropdb --force prodavnica && pg_restore -C -d postgres prodavnica-20261113-154716.dump
  primarna: 21 narudžbi
  replika:  21 narudžbi (replika prati primarnu i kod oporavka)
== 5. RPO i RTO: šta smo izgubili i koliko je trajalo
  izgubljeno: 7 narudžbi = sve upisano nakon zadnjeg backupa
  RPO (od zadnjeg backupa do kvara):       1 s   cilj: <= 3600 s (60 min)
  RTO (od kvara do rada baze):           1.6 s   cilj: <= 900 s (15 min)
  od toga test restore-a:                0.7 s
```

Najvažnija lekcija je u koraku 3: `pg_dump` čuva jednu bazu, ali **ne i uloge**, jer su one globalne za cijelu instancu. Restore u praznu instancu bez uloga `vlasnik`, `app` i `izvjestaj` javlja greške za svaki `OWNER` i `GRANT`, a aplikacija se poslije ne može spojiti. Ovdje ih vraća ista datoteka iz primjera 3, pa je kod za uloge dio plana oporavka, a ne samo udobnost. Provjera nakon restore-a zato ne broji samo redove nego se spaja i kao `izvjestaj`, tj. provjerava i prava.

Brojevi u labu su sekunde jer je baza mala i sve je na istom računaru. Na produkciji RTO čine i otkrivanje kvara, odluka da se radi restore, preuzimanje backupa iz udaljenog storagea (nekoliko stotina GB traje), sam restore s izgradnjom indeksa i preusmjeravanje aplikacije. Zato se test restore-a radi redovno, automatski i na stvarnoj veličini baze, a izmjereno vrijeme se upoređuje s ciljem iz politike: ako ga premašuje, politika se ne ispunjava bez obzira na to što backupi "postoje".

postgresql.conf Isječak 14 · PITR: kontinuirano arhiviranje WAL-a (za usporedbu, nije u labu)

```
archive_mode    = on
archive_command = 'test ! -f /arhiva/wal/%f && cp %p /arhiva/wal/%f'   # u praksi: pgBackRest
ili WAL-G u S3
# restore u trenutak prije greške: fizički backup + WAL do zadanog vremena
restore_command      = 'cp /arhiva/wal/%f %p'
recovery_target_time = '2026-11-20 18:42:00+01'
```

RASPORED I NADZOR

Politika "barem svaki sat" znači i da nešto pokreće backup svaki sat: `cron` ili systemd timer (Ansible moduli `cron` i `systemd_service`), Kubernetes CronJob ili usluga pružatelja. Backup koji tiho prestane raditi je najgori slučaj, jer se otkrije tek kad zatreba. Zato se uz raspored prati i starost zadnjeg uspješnog backupa: alarm kad pređe RPO cilj, npr. metrika "vrijeme zadnjeg backupa" u Prometheusu ili jednostavna provjera datuma objekta u bucketu.

SAŽETAK I PROVJERA ZNANJA

Pregled naredbi, pitanja i zadaci

CILJ	ŠTA RADI	MIJENJA SISTEM?
`make s6-p1` / `s6-p1-lab`	pregled diskova / volumen: mkfs, mount, detach/attach, proširenje	ne / da (loop uređaj, sudo)
`make s6-p2`	Garage: konfiguracija, layout, bucket i ključ, S3 klijent	lokalni proces (127.0.0.1:3900)
`make s6-p3` / `s6-p3-replika`	PostgreSQL: SQL kod 2x i test privilegija / replika i test replikacije	lokalni procesi (127.0.0.1:5461, 5462)
`make s6-p4` / `s6-p4-restore`	backupi s retencijom i kopijom u S3 / kvar, test restore-a, oporavak, RPO i RTO	lokalne datoteke i baza laba
`make s6-clean`	briše sve iz sedmice 6 (procesi, podaci, backupi, volumen)	uklanja

PITANJE	ODGOVOR
Koju vrstu pohrane izabrati za bazu podataka, a koju za backupe?	Bazu na block storage (najmanje kašnjenje, izmjene na mjestu); backupe u object storage (jeftin, neograničen, dostupan s bilo koje lokacije, podržava retenciju i nepromjenjivost).
Zašto se blok volumen ne montira na dvije mašine istovremeno?	Datotečni sistem pretpostavlja da je jedini pisac; dva nezavisna pisca kvare metapodatke. Za dijeljenje služi file storage (NFS, SMB).
Disk je povećan u oblaku, a `df` pokazuje staru veličinu. Zašto?	Povećan je samo blok uređaj; datotečni sistem treba proširiti u OS-u (`resize2fs`, `xfs_growfs`), a ako je na particiji, prvo i particiju (`growpart`).
Šta je "direktorij" u S3?	Ne postoji: kosa crta je dio ključa, a "direktorij" je zajednički prefiks ključeva koji API vraća kao `CommonPrefixes`.
Čemu služi presigned URL?	Privremeno daje pristup jednom objektu za jednu operaciju bez ključa i bez javnog bucketa; potpis i rok važenja su u samom URL-u.
Kako je SQL kod za uloge idempotentan kad nema `CREATE ROLE IF NOT EXISTS`?	`SELECT` vrati naredbu samo ako uloga ne postoji, a `\gexec` je izvrši; `ALTER ROLE` i `GRANT` se mogu ponavljati bez promjene stanja.
Zašto aplikacijska uloga nije vlasnik tabela?	Vlasnik može brisati i mijenjati strukturu; kompromitovana aplikacija s ograničenom ulogom ne može obrisati tabele.
Šta radi replikacijski slot i koji je njegov rizik?	Primarna čuva WAL dok ga replika ne preuzme, pa replika može kasniti bez prekida replikacije; ugašena replika sa slotom puni disk primarne.
Zašto replika nije backup?	Ponavlja svaku izmjenu, i pogrešnu: `DROP TABLE` na primarnoj nestaje i na replici za milisekunde.
Zašto restore u praznu instancu traži kod iz primjera 3?	`pg_dump` ne sadrži uloge (globalne su za instancu); bez njih ne prolaze `OWNER` i `GRANT`, a aplikacija se ne može spojiti.
Kako se smanjuje RPO, a kako RTO?	RPO: češći backup, arhiva WAL-a (PITR), replika za kvar mašine. RTO: automatizovan i uvježban restore, manji i paralelni restore, replika spremna za promociju.

Zadaci za samostalni rad

1. U `s6-p1-lab` umjesto ext4 napraviti XFS (`mkfs.xfs`), proširiti ga (`xfs_growfs /mnt/ruo6`) i pokušati smanjiti. Opisati razliku.
2. Na Linux VM-u iz sedmice 2 postaviti NFS server (`nfs-kernel-server`, `/etc/exports`) Ansible rolom i montirati izvoz s računara. Pokazati da datoteku napisanu s jedne strane odmah vidi druga, i šta mijenja `root_squash`.
3. U `s3.py` dodati naredbu koja ispisuje ukupnu veličinu svih objekata pod zadanim prefiksom (paginacija: `list_objects_v2` vraća najviše 1000 ključeva) i napraviti drugi ključ koji smije samo čitati bucket (`garage bucket allow --read`). Pokazati da upis s njim pada.
4. U `sql/` dodati migraciju `05-placanja.sql` s tabelom `prodaja.placanja`. Bez izmjene `04-privilegije.sql` pokazati da `app` u nju smije pisati, i objasniti zašto. Predati otisak stanja nakon dvije primjene.
5. Dodati ulogu `analiticar` koja smije čitati `prodaja.narudzbe`, ali samo kolone `iznos` i `vrijeme` (privilegije na nivou kolone), i to samo na replici (savjet: `pg_hba.conf` i drugi port). Testom pokazati šta smije, a šta ne.

6. Promijeniti politiku na `ZADRZI=5` i `RPO_CILJ_MIN=1`, napraviti cron red koji ispunjava taj RPO i pokrenuti `make s6-p4-restore` nakon 3 minute bez backupa. Predati izlaz i objasniti zašto je politika prekršena.
7. Uključiti arhiviranje WAL-a na primarnoj instanci (isječak 14), napraviti `pg_basebackup`, upisati nekoliko narudžbi, zabilježiti vrijeme, obrisati tabelu i vratiti instancu u trenutak prije brisanja. Uporediti izgubljene podatke s primjerom 4.

PROJEKTNI ZADATAK

Projekat traži da podaci aplikacije imaju vlasnika i plan: baza s ulogama i privilegijama iz koda (ne superuser za aplikaciju), backup po napisanoj politici s kopijom u object storage, te dokumentovan i ponovljiv test restore-a s izmjerenim RPO i RTO. Na usmenoj odbrani se očekuje da se restore pokaže uživo.

SLJEDEĆA SEDMICA

Podaci su sigurni, ali aplikacija i dalje radi na jednoj adresi. Sedmica 7 uvodi load balancing i visoku dostupnost: HAProxy i Nginx s health checkovima, keepalived/VRRP za plutajuću adresu, auto-scaling, blue/green i canary deployment, te predstavljanje i predaju projektnog zadatka.